STRIPE PROJECT

1. OLTP Data System

Annexes

1.A. OLTP Conceptual Data Model (CDM)

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / OLTP Data System - Annexe A / v30 - 2026/10/13.*

Ce document présente le modèle conceptuel du modèle de données transactionnelles *Stripe*.

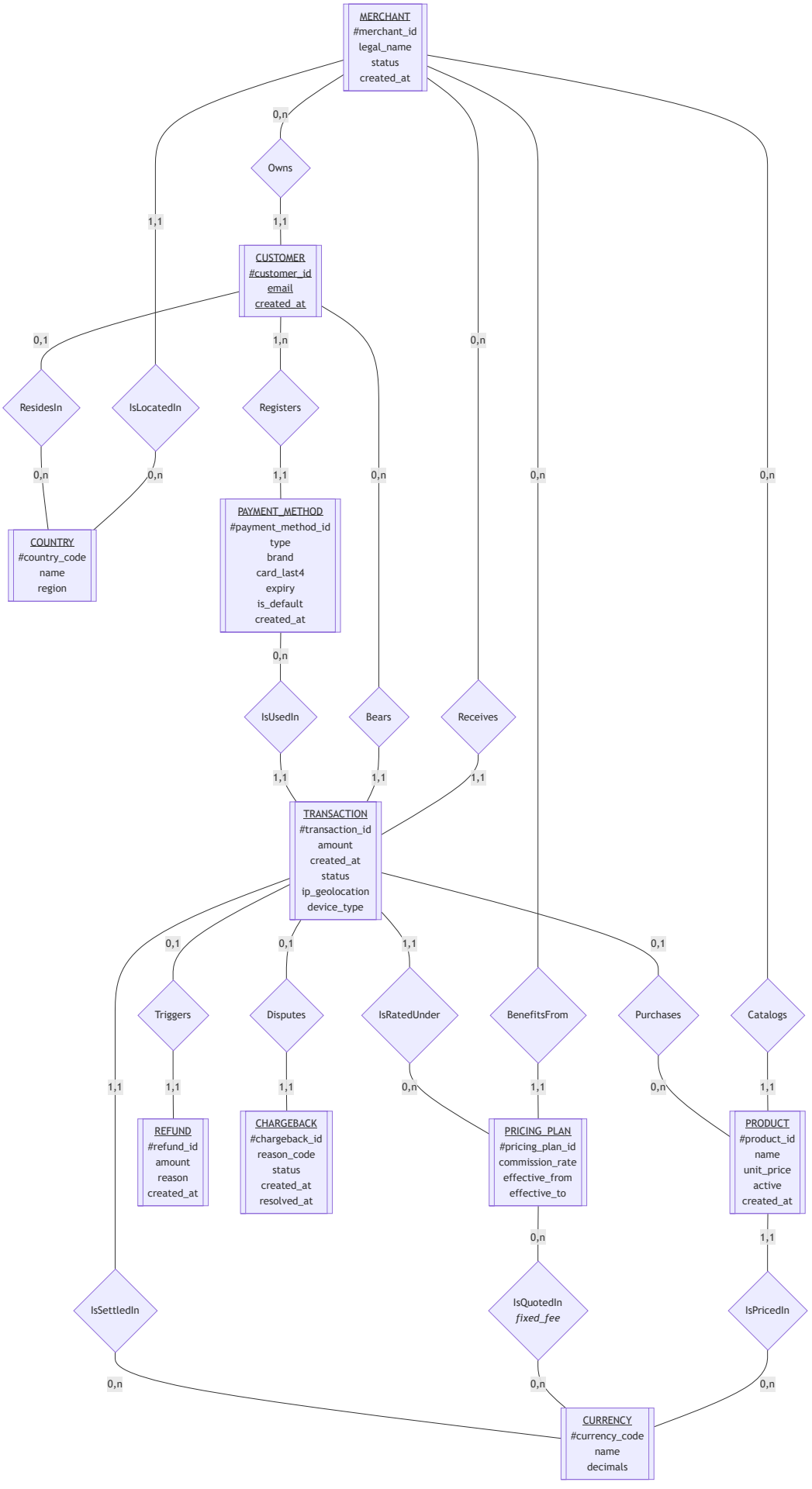
L'approche *Merise* permet de découvrir le système conceptuel entités-associations, de le formaliser et de le transformer en modèle logique. Cette approche s'attache d'abod à décrire le métier indépendamment de toute implémentation physique : dans le modèle conceptuel, on n'a pas de clé étrangère, de typage SQL, ou de table de jointure. Ces derniers éléments n'entrent en jeu qu'au moment de la transformation du modèle conceptuel en modèle *logique* (cf. §1.A.5). Le modèle logique est présenté *en creux* via l'identification des différences entre les deux modèles.

1.A.1. Diagrammes

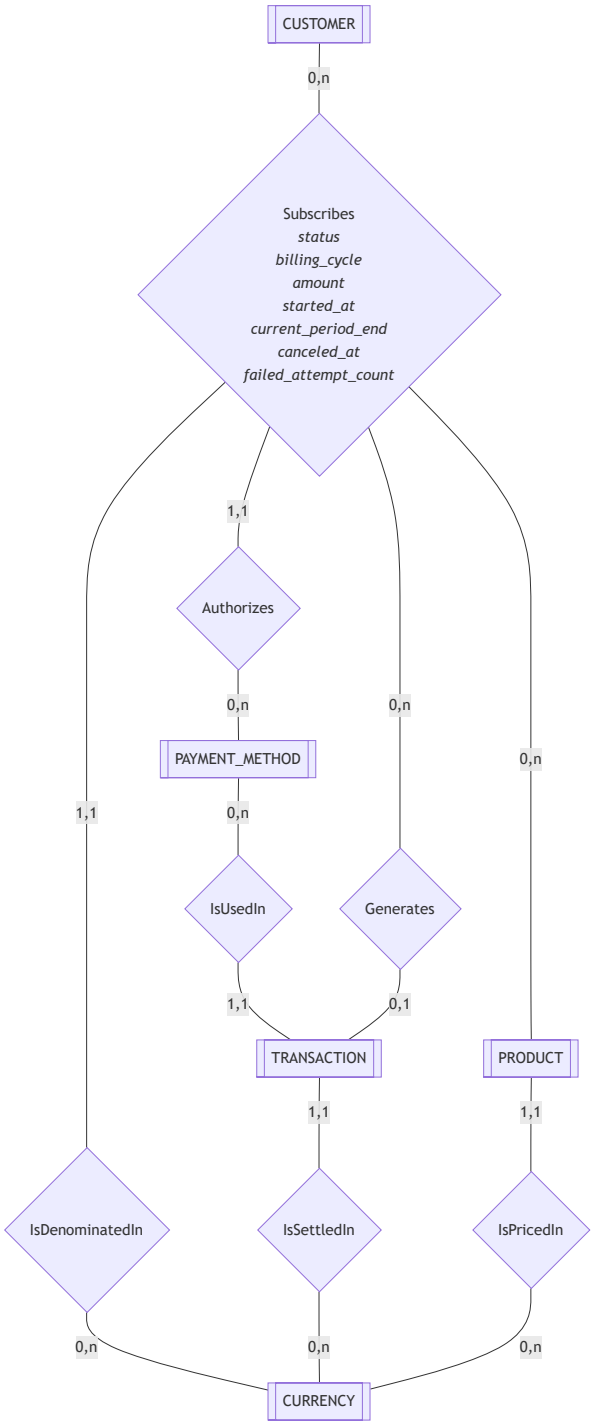
Le modèle conceptuel est schématisé par les diagrammes suivants : vue du coeur de métier, vue du scoring de fraude, vue de sécurité et de conformité. Les rectangles représentent des *entités* et les losanges des *associations*. Les cardinalités sont inscrites sur les liens au format `(min, max)`.

1.A.1.1. Vues du coeur de métier

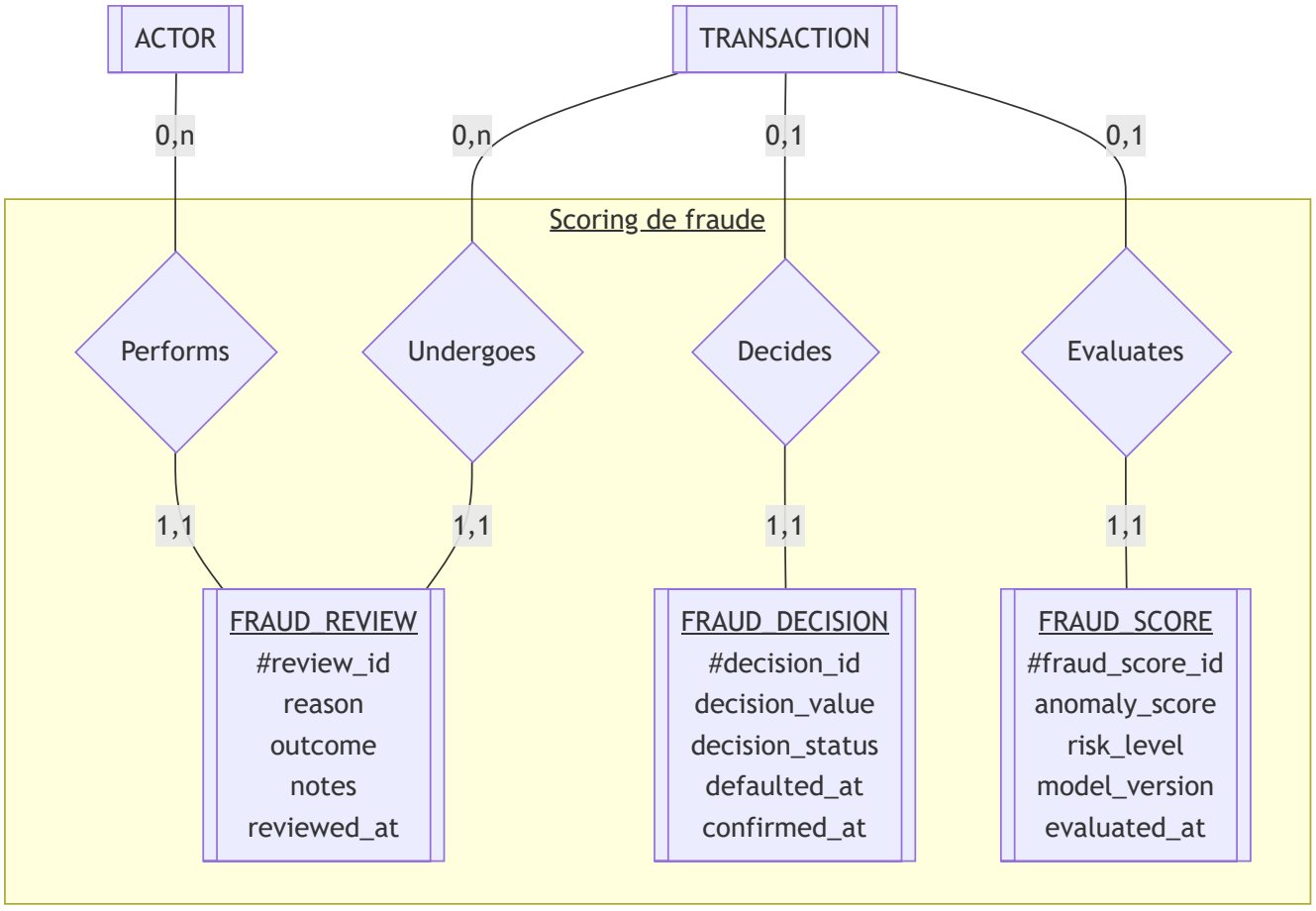
Transactions standards (hors abonnements)



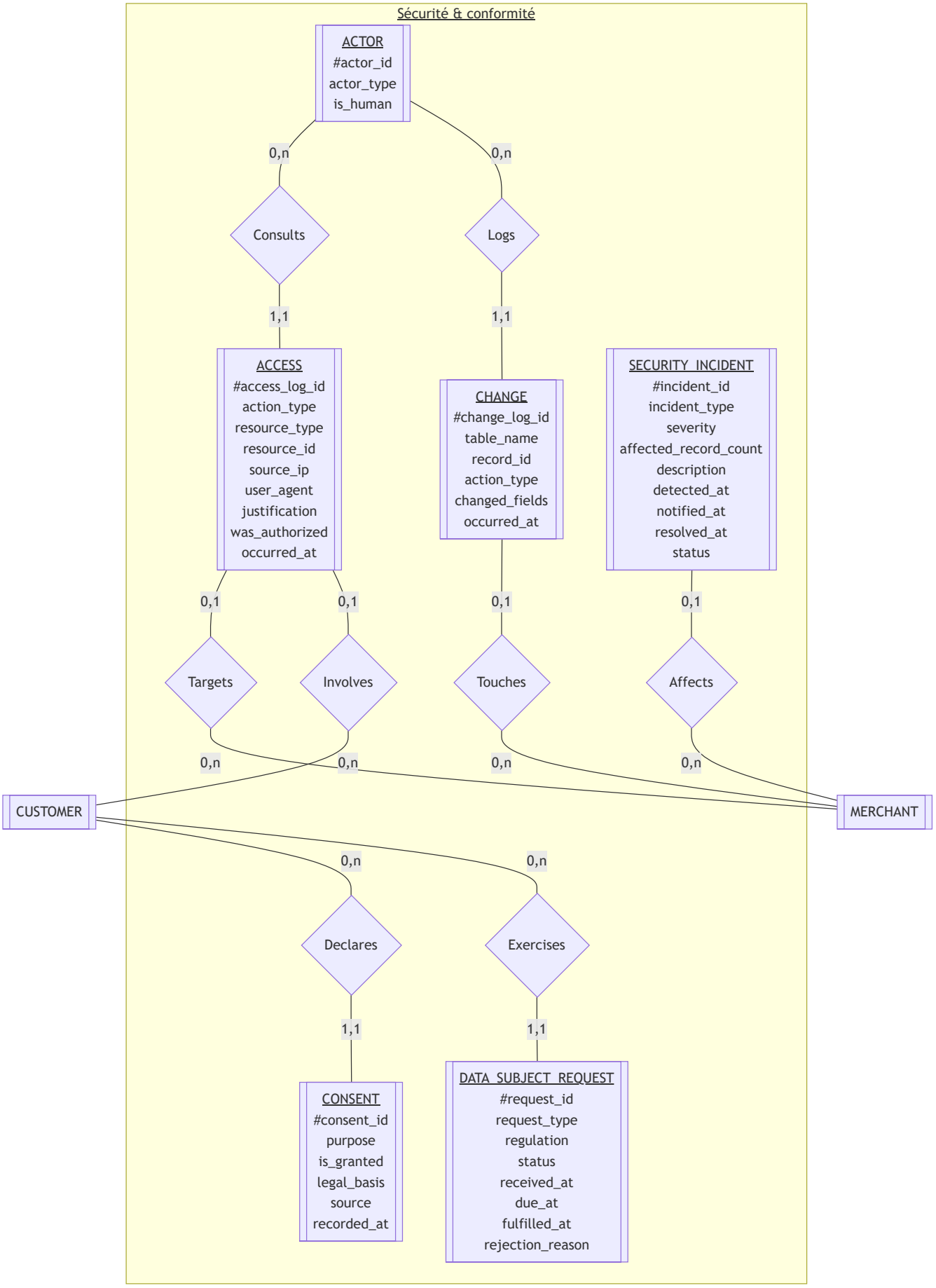
Transactions automatiques (via abonnements)



1.A.1.2. Vue du scoring de fraude



1.A.1.3. Vue de sécurité & de conformité



1.A.2. Dictionnaire des entités

Le symbole `#` marque l'identifiant.

Entité	Propriétés
COUNTRY	#country_code, name, region
CURRENCY	#currency_code, name, decimals
MERCHANT	#merchant_id, legal_name, status, created_at
CUSTOMER	#customer_id, email, created_at
PAYMENT_METHOD	#payment_method_id, type, brand, card_last4, expiry, is_default, created_at
PRODUCT	#product_id, name, unit_price, active, created_at
TRANSACTION	#transaction_id, amount, created_at, status, ip_geolocation, device_type
FRAUD_SCORE	#fraud_score_id, anomaly_score, risk_level, model_version, evaluated_at
FRAUD_REVIEW	#review_id, reason, outcome, notes, reviewed_at
FRAUD_DECISION	#decision_id, decision_value, decision_status, defaulted_at, confirmed_at
REFUND	#refund_id, amount, reason, created_at
CHARGEBACK	#chargeback_id, reason_code, status, created_at, resolved_at
PRICING_PLAN	#pricing_plan_id, commission_rate, effective_from, effective_to
ACTOR	#actor_id, actor_type, is_human
ACCESS	#access_log_id, action_type, resource_type, resource_id, source_ip, user_agent, justification, was_authorized, occurred_at
CHANGE	#change_log_id, table_name, record_id, action_type, changed_fields, occurred_at
CONSENT	#consent_id, purpose, is_granted, legal_basis, source, recorded_at
DATA_SUBJECT_REQUEST	#request_id, request_type, regulation, status, received_at, due_at, fulfilled_at, rejection_reason
SECURITY_INCIDENT	#incident_id, incident_type, severity, affected_record_count, description, detected_at, notified_at, resolved_at, status

1.A.3. Dictionnaire des associations

Association	Entité A	Card. A	Entité B	Card. B	Propriétés portées
IsLocatedIn	MERCHANT	1,1	COUNTRY	**0,n**	-
ResidesIn	CUSTOMER	0,1	COUNTRY	0,n	-
Owns	MERCHANT	**0,n**	CUSTOMER	1,1	-
Catalogs	MERCHANT	**0,n**	PRODUCT	1,1	-
IsPricedIn	PRODUCT	1,1	CURRENCY	**0,n**	-
Registers	CUSTOMER	1,n	PAYMENT_METHOD	1,1	-
Subscribes	CUSTOMER	0,n	PRODUCT	0,n	status , billing_cycle , amount , started_at , current_period_end , canceled_at , failed_attempt_count
Authorizes	SUBSCRIBES	1,1	PAYMENT_METHOD	0,n	-
IsDenominatedIn *(ajouté v22)*	SUBSCRIBES	1,1	CURRENCY	0,n	-
BenefitsFrom	MERCHANT	0,n	PRICING_PLAN	1,1	-
IsQuotedIn	PRICING_PLAN	0,n	CURRENCY	0,n	fixed_fee
Receives	MERCHANT	**0,n**	TRANSACTION	1,1	-
Bears	CUSTOMER	0,n	TRANSACTION	1,1	-
IsUsedIn	PAYMENT_METHOD	0,n	TRANSACTION	1,1	-
IsSettledIn	TRANSACTION	1,1	CURRENCY	**0,n**	-
IsRatedUnder	TRANSACTION	1,1	PRICING_PLAN	0,n	-
Generates	SUBSCRIBES	0,n	TRANSACTION	0,1	-
Purchases	TRANSACTION	0,1	PRODUCT	0,n	-
Triggers	TRANSACTION	0,1	REFUND	1,1	-
Disputes	TRANSACTION	0,1	CHARGEBACK	1,1	-
Evaluates	TRANSACTION	0,1	FRAUD_SCORE	1,1	-
Decides	TRANSACTION	0,1	FRAUD_DECISION	1,1	-
Undergoes	TRANSACTION	0,n	FRAUD_REVIEW	1,1	-
Performs	ACTOR	0,n	FRAUD_REVIEW	1,1	-
Consults	ACTOR	0,n	ACCESS	1,1	-
Logs	ACTOR	0,n	CHANGE	1,1	-
Targets	ACCESS	0,1	MERCHANT	0,n	-
Involves	ACCESS	0,1	CUSTOMER	0,n	-
Touches	CHANGE	0,1	MERCHANT	0,n	-
Declares	CUSTOMER	0,n	CONSENT	1,1	-
Exercises	CUSTOMER	0,n	DATA_SUBJECT_REQUEST	1,1	-
Affects	SECURITY_INCIDENT	0,1	MERCHANT	0,n	-

1.A.4. Règles de gestion

Les règles de métier suivantes gouvernent les relations entre entités :

- **#1. Localisation d'un marchand.** Un *marchand* est établi dans un, et un seul *pays* ; un *pays* peut accueillir plusieurs *marchands*, ou aucun.
- **#2. Relations client/marchand.** Un *client* appartient à un *marchand*, et un seul.
    - > Une même personne physique achetant chez plusieurs marchands correspond à plusieurs occurrences de *client* : le modèle conceptuel modélise les relations *client-marchand* et non les personnes physiques.
- **#3. Moyen de paiement d'un client.** Un *client* enregistre au moins un *moyen de paiement* ; un *moyen de paiement* appartient à un *client*, et un seul.
- **#4. Relation marchand/produit.** Un *produit* appartient au catalogue d'un *marchand*, et unb seul ; un *marchand* catalogue zéro, un ou plusieurs *produits*.
- **#5. Relation produit/devise.** Un *produit* est tarifé dans une seule *devise*.
- **#6. Relations client/produit/abonnement.** Un *client* peut *souscrire* plusieurs *produits*, et un *produit* peut être *souscrit* par plusieurs *clients*. L'*abonnement* (la souscription) est un service du *marchand* ; *Stripe* en fournit l'infrastructure.
    - > La relation `Subscribes` est de type *many-to-many* et porte ses propres propriétés.
- **#7. Transaction et produits.** Une *transaction* peut porter sur un *produit* du catalogue, ou être un paiement libre.
    - > La relation `Purchases` est de cardinalité 0,1 des deux côtés.
- **#8. Origine d'une transaction.** Une *transaction générée* résulte d'un prélèvement automatique déclenché par le *marchand* dans le cadre d'un *abonnement*. Une *transaction non générée* résulte d'une action directe du *client* (achat par le catalogue ou paiement libre).
    - > Une transaction est *générée* si elle est reliée à une occurrence de `Generates`.
    - > Une transaction est *non générée* si elle n'est reliée à aucune occurrence de `Generates`.
    - > Les différents types d'initiative de transaction (générée/non générée) ne sont pas stockés comme attribut au niveau conceptuel : ils se déduisent de la présence ou de l'absence de `Generates`.
        - > Cette règle couvre entièrement l'attribut de transaction `is_merchant_initiated` introduit dans le modèle logique (réf.?) : aucun attribut conceptuel supplémentaire n'est nécessaire, la donnée logique est un booléen dérivé de ce lien.
- **#9. Transaction/marchand.** Une *transaction* est destinée à un *marchand*, et une seul ; un *marchand* reçoit zéro, une ou plusieurs *transactions*.
- **#10. Transaction/client.** Une *transaction* est initiée par un *client*, et un seul ; un *client* est initie zéro, une ou plusieurs *transactions* (soit personnellement, soit automatiquement via les souscriptions).
- **#11. Transaction/moyen de paiement.** Une *transaction* est réglée par un moyen de paiement, et un seul ; un moyen de paiement peut régler zéro, une ou plusieurs transactions.
- **#12. Transaction/devise.** Une *transaction* est libellée dans une *devise*, et une seule ; une *devise* règle zéro, une ou plusieurs *transactions*.
- **#13. Remboursement.** Une *transaction* donne lieu à un *remboursement* au plus ; un *remboursement* se rapporte obligatoirement à une *transaction*, et une seule (le *remboursement* est une entité dépendante, qui ne peut exister sans *transaction*).
- **#14. Litige.** Une *transaction* donne lieu à un *litige* au plus ; un *litige* se rapporte obligatoirement à une *transaction*, et une seule (c'est une entité dépendante, comme le *remboursement*).
- **#15. Score de fraude.** Une *transaction* fait l'objet d'une *évaluation de fraude* au plus ; un *score de fraude* se rapporte obligatoirement à une seule transaction : entité dépendante.
- **#16. Décision de fraude.** Une *transaction* fait l'objet d'une décision de fraude, au plus ; une décision de fraude se rapporte obligatoirement à une *transaction*, et une seule (c'est une entité dépendante, comme le *remboursement* et le

litige).

- **#17. Existence du marchand.** Un *marchand* peut exister sans avoir de *client*, de *produit*, de *transaction* ou de *plan tarifaire* (exemple: cas d'un marchand nouvellement intégré, en cours de vérification).
- **#18. Existence des données de référence.** Un *pays* ou une *devise* peuvent figurer dans le référentiel sans qu'aucun *marchand*, *produit* ou *transaction* ne les utilise : ce sont des données de référence, dont l'existence est indépendante de leur usage.
- **#19. Cohérence entre souscription et moyen de paiement** Une *souscription* est obligatoirement autorisée sur un seul *moyen de paiement* (celui qui sera débité à chaque échéance) ; un même *moyen de paiement* peut autoriser plusieurs *souscriptions*, ou aucune. Le *moyen de paiement* autorisé pour une *souscription* appartient obligatoirement au même *client* que celui qui a souscrit.

    - C'est une contrainte de cohérence entre deux chemins du modèle (`Subscribes` → `Customer` d'une part, `Authorizes` → `PaymentMethod` → `Registers` → `Customer` d'autre part), que les cardinalités seules ne peuvent pas exprimer. Elle devra être vérifiée par une contrainte applicative ou un contrôle d'intégrité au niveau logique/physique (par exemple une contrainte `CHECK` /trigger comparant `Subscription.customer_id` et `PaymentMethod.customer_id`).

- **#20.** `canceled_at` **vs.** `current_period_end` **.** Ces deux informations sont indépendantes et ne doivent pas être confondues : la première marque le moment où le *client* a demandé l'arrêt de l'*abonnement*, la seconde marque la fin de la période déjà facturée. Une *souscription* résiliée reste *active* (au sens de `status`) jusqu'à la date `current_period_end` : le *client* continue de bénéficier du service qu'il a déjà payé, même après avoir demandé la résiliation. `canceled_at` reste vide tant qu'aucune demande de résiliation n'a été faite, y compris pour une *souscription* qui prend fin par un autre mécanisme (échec de prélèvement répété, par exemple).
- **#21. Versionnement du plan tarifaire.** Un *marchand* peut bénéficier de plusieurs *plans tarifaires* successifs au fil du temps (renégociation de commission, changement de catégorie de risque...) ; à une date donnée, un seul *plan* est en vigueur pour un *marchand* donné.

    - Cette absence de chevauchement entre les périodes de validité (`effective_from` / `effective_to`) de deux plans d'un même *marchand* n'est pas exprimable par les cardinalités seules : elle devra être vérifiée par une contrainte au niveau logique/physique.

- **#22. Traçabilité du plan facturé.** Le *plan tarifaire* réellement appliqué à une *transaction* est théoriquement dérivable du *marchand* récepteur et de la date de la transaction (celui dont la période de validité couvre `created_at`) ; il est néanmoins rendu explicite par une association dédiée, pour garantir la traçabilité historique du calcul de la commission - y compris lorsqu'un *plan tarifaire* est corrigé rétroactivement.
- **#23. Cohérence entre un plan facturé et le marchand bénéficiaire.** Le *plan tarifaire* facturé à une *transaction* doit obligatoirement être un *plan* dont bénéficie le *marchand* qui a reçu cette *transaction*.

    - C'est une contrainte de cohérence entre deux chemins du modèle, du même type que la règle #19, à vérifier par une contrainte d'intégrité au niveau logique/physique plutôt que par les cardinalités.

- **#24.** `fee_amount` **: une valeur dérivée non matérialisée.** Le montant de la *commission Stripe* n'est pas conservé comme attribut au niveau conceptuel ; C'est une valeur dérivée, dans le même esprit que la règle #8 pour `is_merchant_initiated` .

    - *commission Stripe*(*transaction*) [dans la devise de la transaction] = `amount` (*transaction*) × `commission_rate` (*plan facturé*) + `fixed_fee` (`currency` (*transaction*))

- **#25.** `commission_rate` **vs.** `fixed_fee` **.** `commission_rate` , en tant que taux, indépendant de toute devise, est un attribut simple du *plan tarifaire*. `fixed_fee` , à l'inverse, est une somme fixe dont la valeur numérique dépend intrinsèquement de la devise : un même *plan* facture un frais fixe différent selon la *devise* de la *transaction*. Le prélèvement fixe, `fixed_fee` , est une propriété de l'association *n,n* entre le *plan tarifaire* et la *devise*. Le prélèvement fixe réellement appliqué à une *transaction* est celui que porte le couple (*plan* facturé, *devise* de règlement de cette *transaction*).

    - C'est encore une contrainte de cohérence entre deux chemins du modèle, à vérifier au niveau logique/physique.

    - Noter qu'un *plan* ne portant pas encore de frais fixe pour une *devise* donnée ne devrait pas pouvoir facturer une *transaction* dans cette *devise*.

- **#26. Devise de la souscription.** Une *souscription* est exprimée dans une *devise* monétaire, et une seule. Le *montant* associé à la *souscription* est toujours accompagné d'un information sur la *devise* associée. La *devise* d'une *souscription* est indépendante des *devises* des *transactions* individuelles générées dans le cadre de l'abonnement.
    - > On n'a pas de contrainte de cohérence pour imposer un changement de devise en cours d'abonnement.
- **#27. Acteurs des journaux d'accès/d'écritures.** Une *entrée dans le journal des accès/écritures* est toujours produite par un *acteur* bien identifié, et un seul. Réciproquement, un *acteur* n'a aucune contrainte sur le nomnbre d'*entrées de journaux* qu'il peut produire.
- **#28. Identité des acteurs.** L'identité d'un *acteur* est gérée par un système d'identification *externe* au périmètre du modèle de données (autrement dit, la source de vérité de l'identité de l'acteur n'appratien pas au modèle).
- **#29. Association des entrées du journal des accès aux clients et aux marchands.** Une *entrée du journal des accès* peut éventuellement désigner un *marchand* et/ou un *client* donnés, mais aucune de ces deux associations n'est obligatoire. Les associations marchand/client sont *presque* indépendantes l'une de l'autre : il existe des accès transverses (typiquement des accès d'administration) qui ne concernent aucun *marchand* ni aucun *client*, des accès qui concernent un *marchand* donné mais pas ses *clients* (ex: export global des données d'un marchand), voire des accès qui ne concernent qu'un *client* donné. Par contre, il existe une contrainte entre *marchand* et *client*ù* lorsque les deux associations existent en même temps (cf. règle #33).
- **#30. Identification de l'entité modifiée dans une entrée du journal des écritures.** Une *entrée du journal des écritures* porte sur une ligne dans une table quelconque du système ; la ligne modifiée est désignée par le nom de la table et par l'identifiant de l'entité modifiée/insérée/supprimée (`record_id`).
- **#31. Association des entrées du journal des écritures aux marchands.** Une *entrée du journal des écritures* concerne un *marchand*, au plus. Elle ne porte jamais sur un *client* de façon isolée (car il n'y a aucune raison de préjuger que, pour chaque table auditée, il existe un chemin de résolution vers un client (ex: *marchand*, *plan tarifaire*, *régulation*, etc.)).
    - > La récupération de l'identifiant du marchand au moment de la création de l'entrée de journal peut être directe (en tant que champ de l'enregistement accédé : *marchand*, *transaction*, *produit*, *plan tarifaire*, etc.) ou indirecte (en tant que champ d'un enregistrement dépendant lié à l'enregisteement accédé : *méthode de paiement* (associée à un *client*, lui-même associé à un *marchand*), etc.).
- **#32. Protecion des journaux en écriture seule.** Le *journal des accès* et le *journal des écritures* sont des journaux en écriture seule, au sens métier : une entrée, une fois créée, n'est jamais modifiée ni supprimée. C'est la condition de validité d'un journal d'audit. Cette distinction n'est pas exprimable par les cardinalités ; elle devra être portée par les droits d'accès au niveau physique (privilèges `UPDATE` / `DELETE` restreints).
- **#33. Cohérence entre client et marchand d'une entrée du journal des accès.** Lorsqu'une *entrée du journal des accès* référence à la fois un *client* et un *marchand*, le *client* visé doit obligatoirement appartenir au *marchand* ciblé (cf. règle #2).
    - > C'est une contrainte de cohérence entre deux chemins optionnels du modèle, de même nature que les règles #19 et #23 : A vérifier par une contrainte d'intégrité au niveau logique/physique plutôt que par les cardinalités seules.
- **#34. Consentement/client.** Un *consentement* se rapporte à un *client*, et un seul ; le *marchand* concerné n'apparaît pas comme lien conceptuel distinct (contrairement au *plan tarifaire* ou au *moyen de paiement* autorisé, le *marchand* d'un *client* est fixe et entièrement déductible via la relation *marchand-client* (règle #2)). Le *consentement* est une entité purement dépendante : elle ne participe à aucune autre association du modèle, et son entité parente, le *client*, peut porter zéro, une ou plusieurs *consentements* au fil du temps.
    - > Le *marchand* d'un *client* pourra être matérialisé comme colonne dénormalisée dans le modèle logique, pour éviter une jointure systématique dans les rapports de conformité (cf. règle #8).
- **#35. Historisation immuable des consentements.** L'enregistrement d'un *consentement* n'est jamais ni modifié ni supprimé: un retrait de consentement de la part du client crée une nouvelle occurrence de *consentement* et n'écrase pas l'occurrence précédent. L'état courant de consentement pour un couple (*client*, *finalité*) donné se lit sur l'occurrence de *consentement* la plus récente.
- **#36. Demande d'exercice de droit/client.** Une demande d'exercice de droit se rapporte à un seul client (`Exercises`) ; même raisonnement que la règle #34 pour l'absence de lien direct au marchand, et même nature de dépendance pure et répétable.
- **#37. Portée d'un incident de sécurité.** Un incident de sécurité peut concerner un marchand précis ou être transverse à la plateforme (`Affects` , 0,1) ; il ne se rapporte à aucun client individuel dans ce modèle — `affected_record_count` reste une grandeur descriptive, sans lien vers les enregistrements concernés eux-mêmes.

- **#38. Entités de workflow mutables.** `DATA_SUBJECT_REQUEST` et `SECURITY_INCIDENT` sont des entités de workflow mutables : leur `status` évolue dans le temps (une demande passe de `received` à `fulfilled`, un incident de `open` à `closed`), et des dates de traitement viennent compléter l'enregistrement initial - à l'inverse des journaux en écriture seule (règle #32).
- **#39. Entités dépendantes avec second rôle.** Certaines entités dépendantes jouent également un second rôle structurel dans le modèle. Dans ce cas, leur dépendance à un parent unique ne les réduit pas à de simples extensions isolées.
  - > exemple 1 : `PRICING_PLAN`, de parent `MERCHANT`, référence `CURRENCY` via `IsQuotedIn` ;
  - > exemple 2 : `ACCESS` (resp. `CHANGE`), de parent `ACTOR`, référence `MERCHANT` et/ou `CUSTOMER` via `Targets`, `Involves` ou `Touches`.
- **#40. Suppression des entités dépendantes.** La suppression d'une entité parente (*transaction,* client*,* marchand *ou* acteur*) entraîne logiquement celle de ses entités dépendantes. La règle ne s'applique pas aux entités de référence (*pays*,* devise*) ou aux entités à rôles multiples (*produit*,* moyen de paiement*) car celles-ci participent chacune à plusieurs associations et ne sont donc pas dépendantes d'un parent unique. **Exceptions** : la règle ne peut pas s'appliquer telle quelle aux instances parentes liées à des entités immuables (p.e. la transaction associée à un score de fraude). Lorsque des entiés dépendantes immuables existent la procédure de suppression doit être remplacée par une procédure d'anonymisation (hors périmètre du présent document).
- **#41. Cycle de vie d'une décision de fraude.** une *décision de fraude* traverse un à deux états successifs (portés par `decision_status`) :
  - elle est créée `confirmed` (décision de scoring confirmée ou infirmée à l'issue d'une revue), et reste dans cet état pour le reste de sa vie ;
  - elle est créée `defaulted` (décision automatique par défaut, en l'absence de revue), et basculée plus tard, éventuellement, dans l'état `confirmed`.
  - > L'absence de *décision de fraude* sur une *transaction* n'est pas un troisième état explicité de `decision_status` ; l'absence de *décision* liée suffit à identifier les *transactions non décidées*.
  - > La *décision de fraude* trace uniquement les décisions fermes (`decision_value` : légitime/fraude) ; elle ne s'occupe pas de la source de cette décision.
- **#42. Dépendance double et répétabilité de la revue de fraude.** Une *transaction* peut faire l'objet de plusieurs *revues de fraude*, ou d'aucune. Une *revue de fraude* est une entité dépendante, de dépendance *double* : elle ne peut exister sans une *transaction* parente et un *acteur* qui la réalise. Une *revue* est déclenchée par un motif (`reason` : score incertain, échantillonnage aléatoire, etc.) et et se termine (`reviewed`) par un résultat (`outcome`). Le résultat d'une *transaction* peut conduire éventuellement à la création d'une *décision de fraude* (cf. #44).
- **#43 Mutabilité de la revue de fraude.** Une *revue de fraude* est mutable : une correction d'erreur de saisie (motif, notes, conclusion) peut se faire directement sur l'occurrence existante, sans qu'il y ait besoin de créer une nouvelle instance de *revue*. Répétabilité (plusieurs revues distinctes par transaction, règle #42) et mutabilité (chaque revue individuelle reste corrigible) sont deux propriétés indépendantes de l'entité.
- **#44. Découplage revue/décision de fraude.** Une *revue de fraude* et une *décision de fraude* associées à la même *transaction* ne sont associées structurellement : une *revue* peut aboutir à une actualisation de la *décision*, ou rester sans suite, et une *décision* peut être posée sans *revue* préalable.

1.A.5. Règles de passage au modèle logique

Configuration	Règle	Exemple dans le modèle
Association *1,n - 1,1*	L'identifiant du côté *1,n* migre en clé étrangère vers l'entité du côté *1,1*	`Owns` → `Customer.merchant_id`
Association *0,1 - 1,1*	Idem, avec une contrainte d'unicité sur la clé étrangère	`Triggers` → `Refund.transaction_id` unique
Association *n,n*	Devient une table à part entière, portant ses propriétés	`Subscribes` → table `Subscription`
Entité de référence	Devient une table ; l'identifiant naturel peut servir de clé primaire	`COUNTRY` → `Country.country_code`
Attribut dérivable d'un lien conceptuel	Matérialisé en colonne lorsque c'est justifié par un gain de performance	`Generates` (présence/absence du lien) → `Transaction.is_merchant_initiated`
Association *1,1 - 0,n* portée par une *pseudo-entité*	La table issue de l'association reçoit directement la clé étrangère (côté *1,1*)	`Authorizes` → `Subscription.payment_method_id`
Propriété portée par une association *n,n*	Devient directement une colonne de la table issue de l'association	`Subscribes.canceled_at` → `Subscription.canceled_at`
Association *0,n - 1,1* avec versionage temporel	L'identifiant du côté *0,n* migre en clé étrangère, accompagné des colonnes de validité déjà portées par l'entité versionnée	`BenefitsFrom` → `PricingPlan.merchant_id` (+ `effective_from` / `effective_to`)
Association *1,1 - 0,n* conservée pour traçabilité malgré une dérivabilité théorique	La clé étrangère est matérialisée explicitement au MLD/MPD, même si elle serait recalculable, pour figer l'historique	`IsRatedUnder` → `Transaction.pricing_plan_id`
Valeur dérivée d'une association et des attributs qu'elle relie	Peut être matérialisée en colonne calculée au MLD/MPD pour la performance, sans exister comme attribut au MCD	`IsRatedUnder` + `PRICING_PLAN.commission_rate` + `IsQuotedIn.fixed_fee` → `Transaction.fee_amount` (OLTP), `FactTransaction.fee_amount_eur` (OLAP)
Association *n,n* portant une propriété dépendante des deux entités reliées	Devient une table de jonction à part entière	`IsQuotedIn` → table `PricingPlanFee` (`pricing_plan_id`, `currency_code`, `fixed_fee`)
Attribut dérivable via une association tierce, matérialisé pour éviter une jointure	Le lien conceptuel n'existe pas ; l'attribut est dupliqué	`Owns` (client → marchand) → `ConsentRecord.merchant_id`, `DataSubjectRequest.merchant_id`
Entité conceptuelle sans table dédiée	Reste un simple attribut texte lorsque la gestion de l''identité est externe au système	`ACTOR` → `DataAccessLog.actor_id` / `ChangeAuditLog.actor_id` (texte, sans table `Actor`)

1.A.6. Différences entre les modèles conceptuel et logique

1.A.6.1. La notion d'abonnement est une entité logique mais pas conceptuelle

La relation entre `CUSTOMER` et `PRODUCT` est une association **many-to-many** enrichie d'informations propres. L'entité `Subscription` du modèle logique n'existe pas au niveau conceptuel : un abonnement est conceptuellement une relation entre un client et un produit.

L'abonnement en tant qu'entité viendra avec le modèle logique par application de la règle : toute association *n,n* devient une table dont la clé primaire est composée des identifiants des entités reliées - ou, choix d'implémentation courant, reçoit un identifiant propre pour simplifier les références.

`Subscribes` en tant que pseudo-entité conceptuelle.
Le formalisme *Merise* classique ne prévoit pas qu'une association participe directement à une autre association : seules des entités peuvent participer. Ce document déroge à cette règle à deux reprises - `Subscribes` participe à `Generates` (vers `Transaction`) et à `Authorizes` (vers `PaymentMethod`).

Ce choix est assumé : dès qu'une association porte des propriétés propres (`status`, `amount`...), elle a le statut d'un objet de métier identifiable (un abonnement), et il est plus lisible de la traiter comme telle plutôt que de dupliquer `Customer` et `Product` dans deux associations ternaires séparées. Dans le modèle logique cela ne change rien : dans tous les cas `Subscribes` devient la table `Subscription` qui porte naturellement les clés étrangères vers `Transaction` (optionnelle, côté `Transaction`) et vers `PaymentMethod` (obligatoire).

Pourquoi une association ternaire `Customer` - `Product` - `PaymentMethod` ne peut pas rempalcer une pseudo-entité?
Parce que `PaymentMethod` détermine déjà `Customer` de façon fonctionnelle (cf. règle #3).
Intégrer les trois entités dans une même association ternaire aurait introduit une redondance : le client deviendrait lisible par deux chemins différents (directement, ou via le moyen de paiement), avec le risque qu'ils divergent.
Passer par `Authorizes` en associant `Subscribes` à `PaymentMethod` évite ce piège (au prix de la règle de gestion #12, qui énonce explicitement la contrainte de cohérence qu'une association simple ne peut pas capturer).

1.A.6.2. Clés étrangères

`TRANSACTION` ne contient ni `merchant_id` ni `customer_id`. Ces liens sont portés par les associations `Receives` et `Bears`. Les clés étrangères apparaîtront par la règle de transformation : dans une association *1,n - 1,1*, l'identifiant du côté *1,n* migre vers l'entité du côté *1,1*.

1.A.6.3. Table d'énumérés

`TransactionStatus`, `ChargebackReasonCode`, `CardBrand` et les autres tables d'énumérés du modèle physique sont des artefacts d'implémentation destinés à garantir l'intégrité et à porter des métadonnées.
Au niveau conceptuel, `status` est simplement une propriété de `TRANSACTION`.

1.A.6.4. Cardinalité et optionnalité

`Triggers` en *0,1 - 1,1* énonce la règle métier : une transaction *peut* avoir un remboursement, et un remboursement se rapporte *obligatoirement* à une transaction. C'est cette cardinalité *0,1* qui justifie, dans le modèle logique, des tables séparées plutôt que des colonnes optionnelles dans `Transaction`.

1.A.6.5. Le modèle conceptuel de transaction ignore la notion de commission

`TRANSACTION` porte un `amount` unique - celui payé par le client. La répartition entre le revenu du marchand (montant moins commission) et le revenu de Stripe (la commission) est une décomposition analytique, mais ce n'est pas une propriété conceptuelle de la transaction.
Elle apparaît côté OLAP, où les deux mesures cohabitent sur la même ligne de faits. Les *paramètres* de calcul de cette commission (`commission_rate` sur `PRICING_PLAN`, `fixed_fee` sur `IsQuotedIn`) sont bien représentés au niveau conceptuel mais le montant de commission résultant reste une valeur calculée.

1.A.6.6. Modélisation de la commission *Stripe*

La commission prélevée sur chaque transaction prend sa source dans le modèle transactionnel. L'entité `PRICING_PLAN` et les associations `BenefitsFrom` et `IsRatedUnder` modélisent les commissions au niveau conceptuel. Un lien direct est conservé entre `TRANSACTION` et `PRICING_PLAN`. La redondance qui en résulte par les chemins `Receives` / `BenefitsFrom` est assumée et encadrée par la règle de gestion #16 (comme la redondance `Subscribes` / `Authorizes` l'est par la règle #12).

`fixed_fee` dépend de la devise.

Le frais fixe prélevé sur chaque transaction est un montant dépendant de la devise associée à la transaction. On a donc une association *n,n* entre `PRICING_PLAN` et `CURRENCY` : chaque plan peut être réalisé dans plusieurs devises, et chaque devise calcule son prélèvement fixe propre,
sans qu'une conversion de change soit nécessaire. La règle de gestion #28 formalise la contrainte de cohérence qui en résulte.

1.A.6.7. Scoring de fraude : trois entités pour trois réalités métier

L'entité `TRANSACTION` porte trois associations avec le domaine de la fraude : `Evaluates`, `Decides`, `Undergoes`. Ces liens ne sont pas redondants et répondent à trois besoins métier distincts.

`FRAUD_SCORE` : l'évaluation du score de fraude d'une transaction.

Cette entité intervient dans deux utilisations du métier :

- *justifier une instruction manuelle de la transaction* : un opérateur qui instruit une revue a besoin de savoir *sur quelle base* la transaction a été signalée ;
- *justifier une décision de blocage auprès d'un marchand ou d'un régulateur* : nécessite d'identifier sans ambiguïté l'unique évaluation qui a motivé l'action de blocage.

`FRAUD_REVIEW` : l'enregistrement d'une revue de transaction.

Une *revue de transaction* est une procédure métier distincte de l'établissement du score de la transaction. Elle est réalisée par un *acteur* externe identifié (`Performs`), répond à un motif propre (`reason`) et produit une conclusion (`outcome`). Elle peut être justifié par un score de fraude incertain, mais pas nécessairement. Elle peut produire une décision automatique lorsque la fraude est confirmée ou informée, mais pas nécessairement.

Cette entité répond à une exigence réglementaire (RGPD) : le droit à l'explication et à l'intervention humaine sur une décision automatisée. Son rôle est d'enregistrer l'évènement proprement dit.

La trace d'une *revue de transaction* reste corrigible (*mutable*) après son premier enregistrement.

Une *transaction* peut faire l'objet de plusieurs *revues* distinctes.

`FRAUD_DECISION` : la décision de métier ferme.

Cette entité est la réponse structurelle au problème de l'étiquetage différé : le statut de fraude ou de légitimité d'une transaction n'est généralement pas établi au moment de la transaction (il est évalué a priori - par le score de fraude - mais la preuve de statut reste à faire). Le modèle doit être capable de représenter un état « de confirmation à venir » sans bloquer les processus qui ont besoin d'une décision immédiate (blocage automatique, reporting).

`FraudDecision` acte la légitimité ou la fraudulence de la transaction par une décision ferme, unique et indépendante de la manière dont elle a été atteinte : par défaut, faute de revue, ou confirmée à l'issue d'une revue.

Pourquoi trois entités distinctes ?

`FRAUD_SCORE` (production automatique, immuable), `FRAUD_REVIEW` (événement d'instruction humaine, répétable) et `FRAUD_DECISION` (décision ferme, évolutive) sont trois réalités de
nature différente : un calcul, un événement, un état. Les confondre obligerait soit à multiplier les colonnes optionnelles (une revue n'a pas de `model_version`, un score n'a pas de `reviewer_id`), soit à perdre la distinction entre « ce qui a motivé un réexamen » et « ce
qui a été décidé » - la règle de gestion #32 formalise ce découplage.

1.A.6.8. Sécurité et conformité

L'entité `ACTOR` n'a pas de table dédiée dans le modèle logique

Le modèle logique ne comporte pas de table `Actor` (`actor_id` y est un simple attribut texte, référençant une identité gérée ailleurs (SSO, annuaire d'entreprise)).
Le modèle conceptuel introduit malgré tout l'entité `ACTOR`, car conceptuellement, "qui a réalisé cet accès" est un objet métier récurrent au même titre que `CUSTOMER` ou `MERCHANT`, même si, au moment de la transformation en MLD, `actor_id` peut rester une colonne simple sans table de
référence propre, si l'équipe juge que la gestion d'identité est du hors périmètre de la base *Stripe*.

`ACCESS` et `CHANGE` sont deux entités distinctes

Dans le modèle conceptuel comme dans le modèle logique, les entités `ACCESS` et `CHANGE` portent de nombreux attributs différents (`source_ip` , `justification` , `was_authorized` pour l'un ; `table_name` , `changed_fields` pour l'autre). La fusion de ces deux entités en une seule table reste utile dans un contexte analytique (reporting consolidé).

Le lien vers `MERCHANT` n'est pas systématique

Les entités `CONSENT` et `DATA_SUBJECT_REQUEST` n'ont pas besoin de ce lien (le marchand est accessible via l'association `Owns` ; voir règle #20).
Les entités `ACCESS` , `CHANGE` et `SECURITY_INCIDENT` portent ce lien explicitement car l'information n'est pas autrement que directement pour ces trois entités (un accès ou un incident peut être transverse à la plateforme, sans qu'il soit possible de l'associer à un marchand).
